



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Building a system

Francesco Guerra
francesco.guerra@unimore.it

Objectives

- ▶ Characterize the size and complexity issues of a system.
- ▶ Describe the technical and non technical issues in development and support of a system.
- ▶ Demonstrate the concerns in the development and support activities of a large application software, using a payroll system example.
- ▶ Describe the coordination efforts needed for process, product, and people.

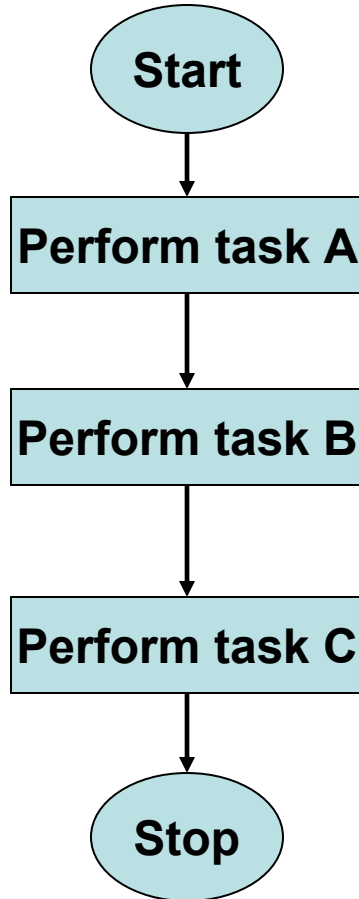
Building a System

- ▶ Software is becoming ubiquitous
 - ▶ The development of systems involving software is becoming complex
- ▶ Problems and concerns associated with building a system that contains multiple components.
 - ▶ From a few components to hundreds or thousands of components
- ▶ Increase in number of components and complexity is what requires us to study
 - ▶ Rationale for Software Engineering as a discipline

Size and Complexity

- ▶ Breadth issues :
 - ▶ Major functions
 - ▶ Features within each functional area
 - ▶ Interfaces to other external systems
 - ▶ Simultaneous users
 - ▶ Types of data and data structures
- ▶ Depth issues
 - ▶ Linkages and relationships among items.
 - ▶ Linkages
 - Sharing of data, and / or sharing of control
 - ▶ Relationships
 - Hierarchical / Sequential / Loop / Recursive / ...

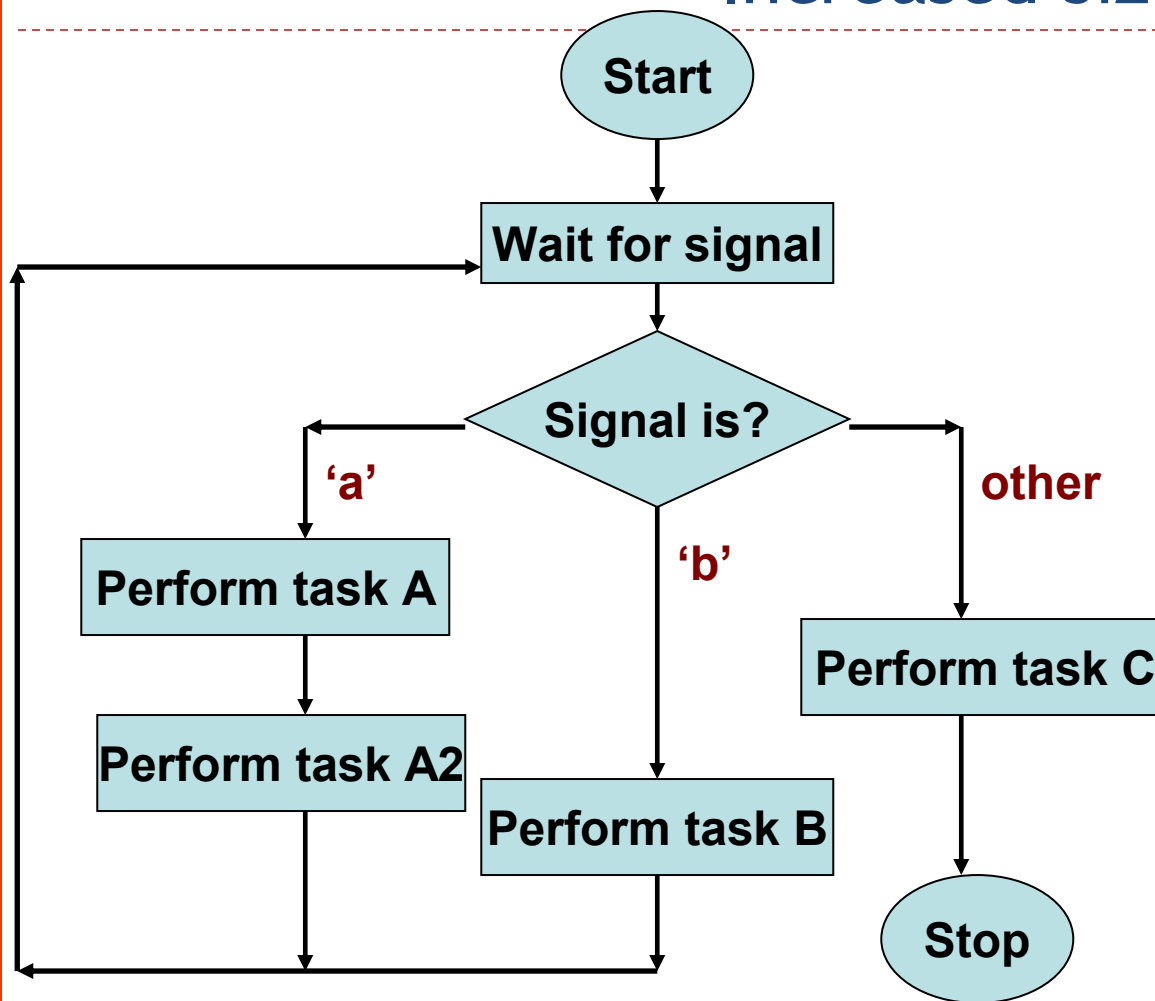
Size and Complexity - Simple case



- ▶ Three major segments
 1. Start the process
 2. Perform three tasks
 3. Stop the process

Size and Complexity

- Increased size and complexity



- ▶ Number of the tasks increased (3→5)
- ▶ New decision task
 - ▶ Greatly increased the number of choices
- ▶ Loop relationship with the decision task

- ▶ A relatively minor increase in the number of tasks and decisions has greatly increased the complexity

Problem and design decomposition

- ▶ Technical issues we must consider moving from simple to complex situations
 - ▶ Concept of “divide and conquer”.
 - ▶ Modularization (Parnas, 1972)
 - ▶ how we divide a complex problem and its solution?
 - ▶ how to handle all pieces, parts and relationships?
- ▶ Modularization
 - ▶ Need to simplify the complex problem by addressing the problem in smaller segments
 - ▶ To decide if we should design and decompose the software system solutions along the dividing lines of the problem segments or according to another dimension (e.g., the objects).

Problem and design decomposition (2)

- ▶ How to choose a decomposition method?
- ▶ The key idea is to consider some form of simplification through the following types of activities:
 - ▶ Decomposition
 - ▶ Modularization
 - ▶ Separation
 - ▶ Incremental iterations
- ▶ Issues concerning design will be addressed later on

Technology and tool considerations

- ▶ The choice of the specific programming language may become an issue in writing large and complex applications
 - ▶ More developers are working in the software
 - ▶ The diversity in experience often results in personal biases for
 - ▶ A certain program language
 - ▶ A certain developing tool
- ▶ A common development language and environment has to be selected
- ▶ Beyond this, other technical choices concern:
 - ▶ Database
 - ▶ Network
 - ▶ Middleware
 - ▶ Other (i.e. code version control)

Process and Methodology

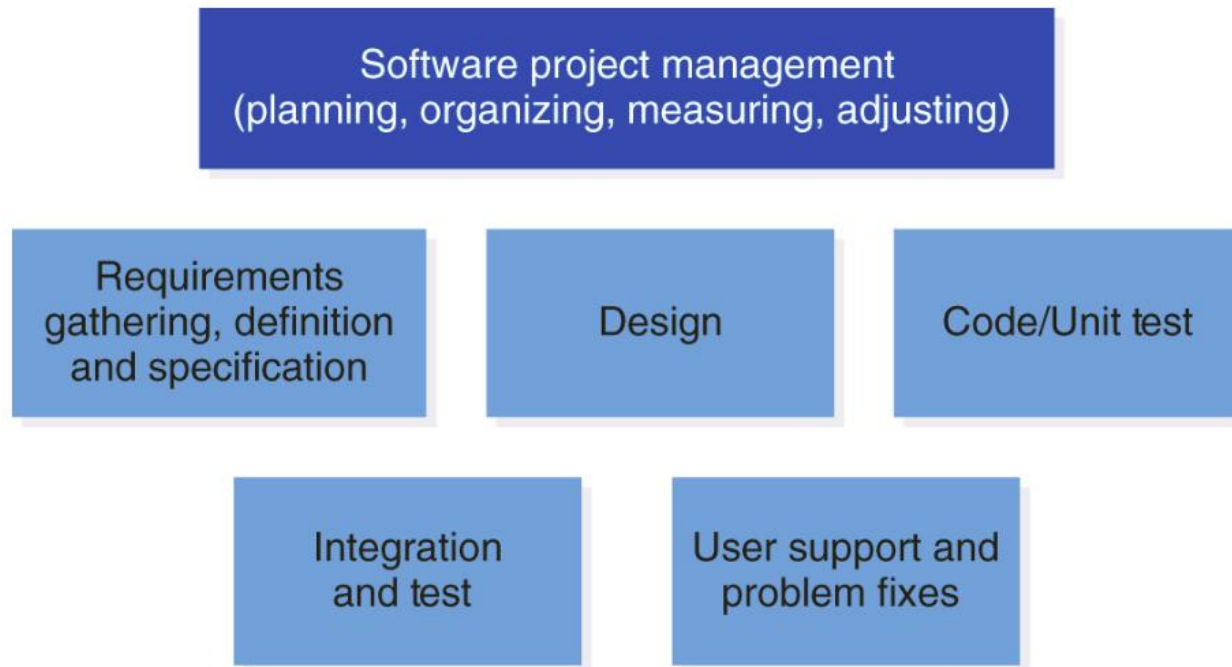
- ▶ If a single person alone is developing a software solution, there is still the need
 - ▶ To understand problem and requirements
 - ▶ To design a solution and to implement it
 - ▶ To test the application (possibly with a user)
- ▶ But
 - ▶ Very little communication between users
 - ▶ No material (e.g., design documents) is passed from people
 - ▶ Need to document the work (not to forget the rationale of the decisions made)
 - ▶ No need to coordinate the work
 - ▶ Required only if 2+ people are involved

Process and Methodology (2)

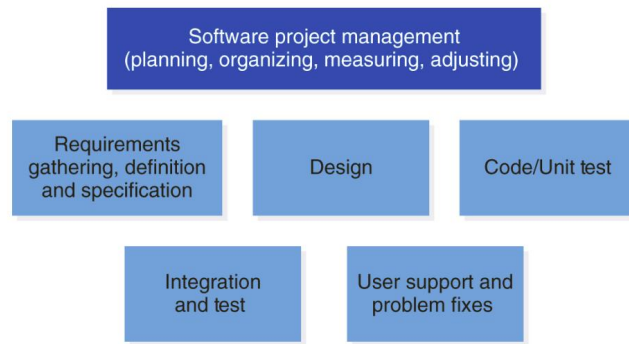
- ▶ In large and complex development situations, a **software development process** is needed to coordinate the group of people
- ▶ Software development process:
 - ▶ the set of tasks / the sequence and flow of these tasks,
 - ▶ the inputs to and the outputs from the tasks,
 - ▶ the pre-conditions and the post-conditions for each of the tasks involved in the production of a software
- ▶ Items to be agreed
 - ▶ Syntax for the expression of the design
 - ▶ Methods for the tasks
- ▶ It is commonly accepted that some process must exist to help coordinate a complex software project

Process and Methodology (3)

- ▶ These are the common tasks performed in software development and support
 - ▶ Each task appears as an independent item
 - ▶ Each one ask the questions of what is expected and how we perform it

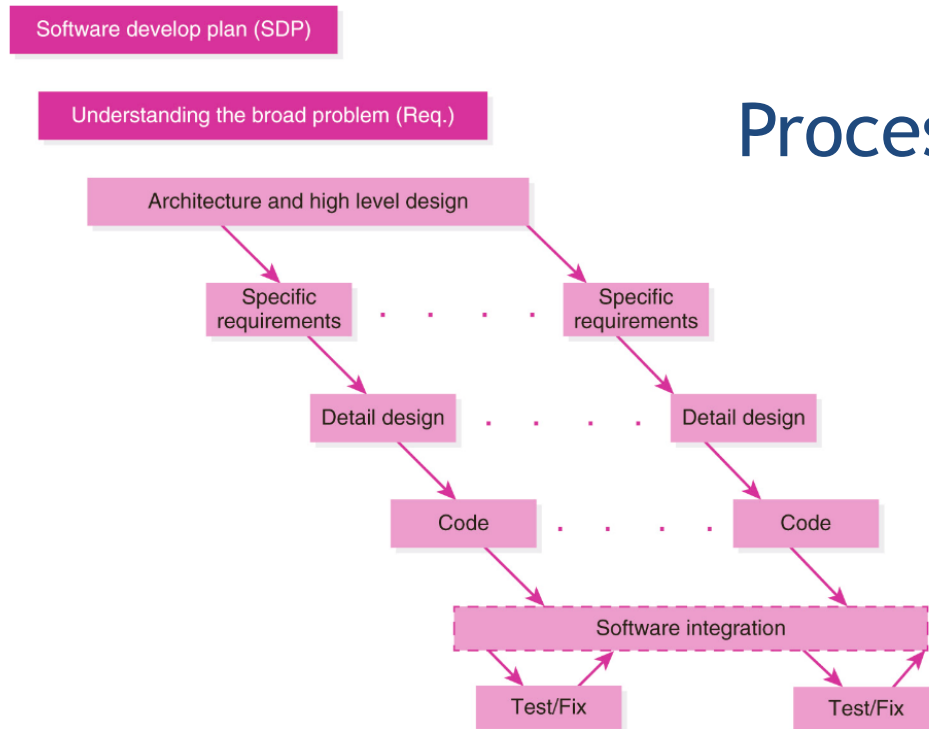


Process and Methodology (4)



- ▶ For example, is there a methodology to gathering the requirements?
 - ▶ If more people are doing this task, we need to define how the process should be broken down
- ▶ The processes are depicted as independent
 - ▶ If several people are involved, a clear understanding of the sequence, overlap and starting conditions is needed
 - ▶ E.g., designers and coders may be a group of people that is different from the requirement analysts who are working with the customers
 - ▶ At which point the designers and coders have to start their work? How much can these tasks overlap?

Process and Methodology (5)



- ▶ The approach in Figure employs the concept of incremental development and continuous integration.
 - ▶ Software integration: the process of linking together individually tested units into a coordinated whole system.
- ▶ **Continuous integration/continuous deployment (CI/CD):** The extension of incremental software development process to include the quick deployment of a completed functionality via (1) continuously integrating completed functionalities into the product, (2) delivering those functionalities to the users, and (3) having the users quickly deploy those functionalities.

Process and Methodology (6)

- ▶ The seemingly simple boxes depicting the test-fix-integrate cycle are extremely deceptive.
- ▶ The process requires a description of a methodology that answers the following questions:
 - ▶ Is there a separate and independent test group?
 - ▶ When a problem is found, how and to whom should it be reported?
 - ▶ How much information must accompany a problem description?
 - ▶ Who decides the severity level of a problem?
 - ▶ How is a problem-fix returned to the tester?
 - ▶ Should all problem-fixes be retested?
 - ▶ How are the problem-fixes integrated back into the code?
 - ▶ What should be done with the problems that are not fixed?
- ▶ The process also assumes that incremental development is used and that both the problem and the design can be decomposed into increments.
- ▶ The Figure does not include the support and customer problem-fix activities.

Nontechnical considerations

- Effort Estimation and Schedule

- ▶ For simple software projects involving 1-3 people the effort estimation and scheduling is relatively easy
 - ▶ Functional and nonfunctional requirements are fewer in number and complexity
- ▶ For large systems, capturing and understanding the requirements can be overwhelming
- ▶ Many projects become failures due to issues to
 - ▶ Estimating the total effort
 - ▶ Generating a reliable project schedule
- ▶ Typically effort is optimistically estimated
 - ▶ Unrealistic expectations on customers

Nontechnical considerations

- Effort Estimation and Schedule (2)

- ▶ Example: a simple software project that requires 3 main functions with a total of 12 functional features
- ▶ The estimation requires a good understanding of
 - ▶ All the functional features
 - ▶ The productivity of the individuals working in the project
- ▶ In large projects, the features can be hundreds or thousands
 - ▶ The probability of understanding all the requirements well and of knowing the productivity of all the individuals is low
 - ▶ The resulting effort estimation is often a good guess and far from being accurate

Nontechnical considerations

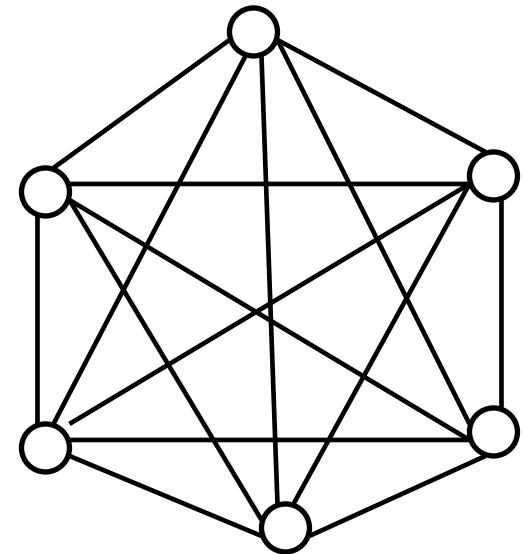
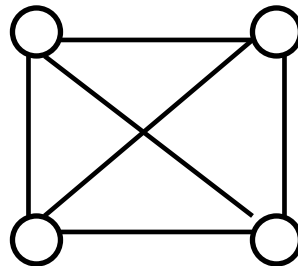
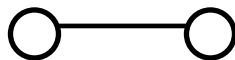
- Assignments and Communications

- ▶ There are activities other than designing and coding that require people resources:
 - ▶ Testing
 - ▶ Integration
 - ▶ Tool support
- ▶ This requires more understanding of the skills of the people involved and the tasks they have to perform
 - ▶ The assignment of the most properly skilled people to the right tasks requires a deeper level of granularity

Nontechnical considerations

- Assignments and Communications (2)

- ▶ Problems of communications
 - ▶ For a small project (2-3 people) the number of communication paths is small
 - ▶ The communication paths increase as the number of participants increases



Nontechnical considerations

- Assignments and Communications (3)

- ▶ Associated with the increase in the number of communication paths
 - ▶ Chance of an error in communication
 - ▶ If the chance of communicating correctly a message between any two people is $2/3$
 - ▶ The probability that we will communicate the message to a third person is $(2/3 * 2/3)$
 - ▶ Such a low probability of correct communication among team members may be a serious problem

Building a Hypothetical Payroll System

- Requirements

- ▶ Possible functional and nonfunctional requirements of a payroll system
 - ▶ Add, modify and delete the names and personal information of all the employees
 - ▶ Add, modify and delete the benefits associated with all employees
 - ▶ Add, modify and delete the tax and deductions associated with all employees
 - ▶ Add, modify and delete the gross income associated with all employees
 - ▶ Add, modify and delete algorithm related to computing the net pay for all employees
 - ▶ Generate a paper check or an electronic bank deposit for each employee

Building a Hypothetical Payroll System

- Requirements (2)

- ▶ Each of the previous functional requirement can be expanded in other levels
 - ▶ E.g., item 1: What is the personal information associated to each employee?
 - ▶ To solve the problem who and where is the person that the software engineer has to ask?
 - ▶ Is the best answer provided by a user, designated official requirements person or project leader?
 - ▶ Should the answer be documented?
- ▶ To properly handle the application we need to understand something about
 - ▶ benefits,
 - ▶ tax law and
 - ▶ other domain specific knowledge

Building a Hypothetical Payroll System

- Requirements (3)

- ▶ The payroll system has to generate the paychecks and deposits
 - ▶ What is the allowable payroll cycle?
 - ▶ If the check and deposit have to be completed by the end of the month, when must the inputs to the cycle (salary increase) have to be closed?
 - ▶ We have to know
 - ▶ The payroll-process-cycle window
 - ▶ The performance capability the system must have to satisfy the window
 - ▶ The software engineer has to know the
 - ▶ Volume of the payroll transaction
 - ▶ Speed of processing each payroll
 - ▶ This means that the software engineer has to know
 - ▶ the hardware and operating system environment capability
 - ▶ Payroll domain knowledge

Building a Hypothetical Payroll System

- Requirements (4)

- ▶ Once the requirements information is documented
 - ▶ The complexity of such a system requires a review with the user /customer prior to passing to the design and coding phase
- ▶ The review can be conducted
 - ▶ gradually if the requirements are incrementally analyzed and documented
 - ▶ All at once when all requirements are analyzed and specified together
- ▶ Either situation requires a coordination of effort between users / customers and the requirements analysts
- ▶ Number of activities extremely high and time consuming

Building a Hypothetical Payroll System

- Requirements (5)

- ▶ A team of requirement analysts can be needed
 - ▶ Individuals with different skills spanning from
 - ▶ Payroll domain specific knowledge
 - ▶ IT and system development expertise
- ▶ From a quality perspective
 - ▶ 15% of software defects are due to requirements errors (Jones 1992)
- ▶ The activities related to completing a requirements specification are difficult and have a significant impact on all downstream activities and on the final product

Designing the Payroll System

- ▶ Once the requirements are agreed and understood, the system has to be designed
- ▶ Requirements incomplete
 - ▶ Should all the “add, updated and delete functions” be grouped into a single component called “payroll administrative function”?
 - ▶ Should all the processing functions (calculation of the deductions, net pay amount) be grouped into a component called “payroll processing”?
 - ▶ For handling errors and exceptions, should we create an “exception-processing component”?
 - ▶ The system must interface with external systems (e.g., direct bank deposits). Should we create a component called “payroll interfaces”?

Designing the Payroll System (2)

- ▶ Grouping related functions into components has some advantages:
 - ▶ Provides some design cohesiveness with the component
 - ▶ Match the business flow and the payroll processing environment
 - ▶ Provides a potential assignment of work by components
 - ▶ Allows easier packaging of the software by components
- ▶ There are possible interactions among the components
 - ▶ Usage of common data / tables

Designing the Payroll System (3)

- ▶ Nonfunctional specific that must be designed
 - ▶ Help service / message service for all the functional components
 - ▶ Services that can be placed in the “services component”
- ▶ Common services
 - ▶ Horizontal components that cross all the individual application features
- ▶ Application domain specific functions
 - ▶ Vertical components
- ▶ The interaction or coupling of the vertical components with the common services is a key design concern
- ▶ During the design the interface layout is finalized
 - ▶ Payroll batch oriented rather than an interactive system
 - ▶ In this case the user interface is not a prime design concern

Designing the Payroll System (4)

- ▶ The design of this payroll system requires a broad set of skills because of the breadth and depth of the system
 - ▶ Breadth perspective
 - ▶ Database / network / transmission interfaces / printing interfaces / OS interfaces / development tools environment / the payroll domain
 - ▶ Depth perspective
 - ▶ The designer needs to understand the specifics of a payroll system (e.g., performance and error processing)
 - ▶ Volume of records in the batch-processing needs special design concerns into the process of handling errors
 - ▶ To catch erroneous information
 - ▶ What should happen to the people whose records cannot be processed?

Code and Unit Testing

- ▶ The high-level design needs to be refined and converted into running code
 - ▶ With each designed component, the functional unit needs to be designed and converted into code
- ▶ For each functional unit the programmer has to address and develop the following material:
 - ▶ Precise layout of the screen interface in some language
 - ▶ Precise functional processing logic in some language
 - ▶ Precise data access and storage logic in some language
 - ▶ Precise interface logic in some language

Code and Unit Testing (2)

- ▶ If there are many programming units we have to define standards
 - ▶ Naming convention for each module (identifying it as the module of a specific component)
 - ▶ Naming convention for data (elements from the same database table should have the same prefix)
 - ▶ Convention about documentation (e.g., providing detail design on the conditions under which module may be entered or exited)
 - ▶ Documentation of how to handle the error conditions
 - ▶ Error messages from different program modules should be consistent

Code and Unit Testing (3)

- ▶ The module are tested to confirm that it performs the intended tasks.
- ▶ Steps:
 - ▶ To set the conditions of the module and to chose the input data
 - ▶ To run the module and observe its behavior (ensuring that the output of the module is performing what it is intended to do)
- ▶ Programming and testing is a heavily human resource intense activity
 - ▶ Large systems are composed of hundreds of modules
 - ▶ The coordination and integration of all the programming efforts is a management challenge

Integration and Functionally Testing the System

- ▶ The modules completed and unit tested have to be collected from the individuals programmers
 - ▶ This activity is known as integration (part of the configuration management activity)
 - ▶ Collections is needed to avoid changes to already tested modules
- ▶ Modules are compiled and linked together to work as a functional unit
- ▶ The functional unit can be part of a bigger component
- ▶ The integrated module is tested with functional test cases generated by a more objective group (not the programmers who coded the modules)

Integration and Functionally Testing the System (2)

- ▶ Functional testing can uncover problems that have to be fixed by the programmers
 - ▶ The cycle problem detection - problem fixed needs to be coordinated
 - ▶ The fix code has to be integrated into the functional units and retested
 - ▶ As a set of modules in a functional unit completes the tests, it is labeled and locked from further changes
- ▶ In a very large system construction, there is usually a tool used to help automate the configuration management mechanism
 - ▶ People and skill required to integrate and testing are different from those needed for coding, designing and requirements gathering
 - ▶ Test scenarios and test scripts often require the knowledge of the requirements and design

Release of the system

- ▶ After the functional units are tested and integrated into components, the components have to be tested together
- ▶ This ensure that the complete system works as a whole
 - ▶ All interfaces across components do work
- ▶ Even after the entire testing through all the test user scenarios in the context of the user business environment, the system cannot be released unless no problem is found
 - ▶ Major problems have to be fixed
- ▶ The tested system has to be managed and protected from further changes

Release of the system (2)

- ▶ The user have also to be educated in the use of the system
 - ▶ For a large system the task has to be planned and orchestrated
- ▶ This requires
 - ▶ The development of training material:
 - ▶ Is a complex task
 - Can involve several people for several months
 - ▶ The deliver of this material requires different skills from technical design or coding
 - It requires communication and presentation skills
 - ▶ The preparation of user support personnel for supporting users in the details of the software and collecting reports on bugs
 - ▶ Support personnel has to be preeducated on the system, user environment and tools needed for supporting users

Release of the system (3)

- ▶ Once
 - ▶ Users have been trained
 - ▶ Support team is trained
- ▶ The system is ready for release to the users
- ▶ Who is the person who makes the final call of a product release?
- ▶ What criteria should that person use in making the determination of the release?

Support and Maintenance

- ▶ For a small (1-2 module) software product used by few people, the support is not a major concern
- ▶ For a large system, the post release support may be a complex task
- ▶ Several assumptions can be made and be included in the calculation of the expected system support effort
- ▶ Many of the following decision factors plays a role:
 - ▶ Number of expected users and customers
 - ▶ Number and type of known problems existing at the release time
 - ▶ Projected number of problems that will be discovered by the users
 - ▶ Amount of users' training
 - ▶ Amount of support personnel training
 - ▶ Number of development personnel committed to support the system
 - ▶ Expected number of problem-fix releases and future functionality releases

Support and Maintenance (2)

- ▶ At least two sets of support personnel with distinct skills are needed to support a complex environment:
 - ▶ A group to answer and handle system usage and simple problem work-arounds
 - ▶ Good communication skills
 - ▶ Good knowledge of the system
 - ▶ A group to fix specific problems and implement future enhancements
 - ▶ Including designers and coders
 - ▶ If it is expected a long product life
 - ▶ The support personnel will resemble a complete development team
 - ▶ The support organization may be comparable in size and complexity to the original developing team

Coordination Efforts

- ▶ Because there are
 - ▶ more parts,
 - ▶ more developers
 - ▶ more users
- ▶ to consider in “Large Systems” than a single program developed by a single person for a limited number of users, there is the need for Coordination of 3P's:
 - ▶ ‘Processes’ and methodologies to be used
 - ▶ Final ‘product’ and intermediate artifacts
 - ▶ ‘People’ (developers, support personnel, and users)

Coordination Efforts - Process

- ▶ Since 80', companies made efforts to prevent, detect and correct problems
 - ▶ Improving the quality of the software
 - ▶ Measurement efforts
 - ▶ Prediction applied to software
 - ▶ Increasing the productivity of the software developers
 - ▶ Supporting software development tools

Coordination Efforts - Product

- ▶ A software product is often thought of as only containing the executable code
- ▶ In developing and supporting software many more artifacts have to be produced
 - ▶ Requirement documents
 - ▶ Test scenarios
 - ▶ User manuals
 - ▶ ...
- ▶ Simply delivering a package of executable software is not enough in many cases

Coordination Efforts - Product (2)

- ▶ Users can ask for design documents and source code (for further modifications)
- ▶ The software product expected to have a long life span is needed to be designed for changes and easy maintenance
- ▶ All good software design will go for high cohesion and low coupling
 - ▶ It is difficult to maintain it when a software product evolves through many cycles of modifications
 - ▶ A software product needs to be protected from design drift and erosion (TMD 2009)

Coordination Efforts - People

- ▶ For a large software a team of people with an assortment of skills and experience is needed
 - ▶ The human element is a crucial component in the development and support of software
- ▶ The study of software engineering must include the issues of coordinating people's activities and management skills



References

- ▶ Frank Tsui, Orlando Karam, Barbara Bernal, Software engineering (Fifth Edition), Jones & Bartlett Learning

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.